

DVWA Command Injection: Comprehensive Walkthrough & Exploit Report

Author: Ibrahim Al-ameen (CYBERHIM) | Target: Damn Vulnerable Web Application (DVWA)

This technical report documents the step-by-step exploitation and analysis of the **Command Injection** module within the Damn Vulnerable Web Application (DVWA). Across Low, Medium, and High security configurations, this documentation covers the mechanics of insecure system calls, blacklist filter evasion, and secure coding remediation.

1. Executive Summary & Core Concepts

OS Command Injection occurs when an application passes unsafe user-supplied data directly to a system shell using execution functions like `shell_exec()`. In this laboratory environment, the target module simulates a network diagnostic tool ("Ping a device") that accepts an IP address from the user.

The Restaurant Analogy: Think of the web application as an obedient robot at a restaurant. If the kitchen does not check the customer's note properly, typing malicious instructions alongside a normal request causes the robot to execute both sequentially.

2. Low Security Analysis & Exploitation

Source Code Review

At the Low security level, the PHP backend contains zero input sanitization or validation checks:

```
$target = $_REQUEST[ 'ip' ];
$cmd = shell_exec( 'ping -c 4 ' . $target );
echo "<pre>{$cmd}<pre>";
```

Exploitation Steps

Because input is concatenated directly into the shell command string without validation, standard Linux command separators (such as semicolons `;` or double ampersands `&&`) can be leveraged to chain secondary commands.

- **Reference Screenshot:** DVWA_SCREENSHOTS/Easy.png
- **Payload Used:** `127.0.0.1 ; cat /etc/passwd`
- **Outcome:** The system pings the loopback address and immediately executes the `cat` utility, dumping the system user database (the public user account file) onto the interface.

3. Medium Security Bypass & Filter Evasion

Source Code Review

At the Medium security level, the developer implemented a basic blacklist to strip out specific malicious symbols:

```
$substitutions = array(
    '&&' => '',
    ';'  => '',
);
$target = str_replace( array_keys( $substitutions ), $substitutions, $target );
```

Exploitation Steps

Blacklists are inherently flawed because they attempt to predict malicious entries rather than allowing safe ones. The filter failed to account for alternative Linux command chaining operators, specifically the single pipe character |.

- **Reference Screenshot:** DVWA_SCREENSHOTS/MEDIUM.PNG
- **Payload Used:** 127.0.0.1 | cat /etc/passwd
- **Outcome:** The payload bypasses the ampersand and semicolon filter, allowing the pipe to push the ping output into the password viewer utility. Alternatively, nested filter evasion payloads like 127.0.0.1 && cat /etc/passwd successfully reconstruct forbidden characters during single-pass evaluation.

4. High Security Evasion & Developer Flaws

Source Code Review

At the High security level, a much more comprehensive blacklist array is introduced to strip multiple characters and operators:

```
$substitutions = array(
    '| ' => '|',
    '&' => '&',
    ';' => ';',
    '| ' => '|',
    '- ' => '-',
    '$ ' => '$',
    '(' => '(',
    ')' => ')',
    '` ' => '`',
);
```

Exploitation Steps

Despite increased strictness, human error in syntax design left a critical flaw. The filter targeted ' | ' (a pipe symbol followed by a literal space), rather than the pipe symbol globally.

- **Reference Screenshot:** DVWA_SCREENSHOTS/Hard.png
- **Payload Used:** 127.0.0.1|cat /etc/passwd
- **Outcome:** By completely omitting the space after the pipe operator, the filter rule failed to match, permitting the command execution sequence to pass through to the underlying shell.

5. Remediation & Secure Coding Principles

Security Level	Defense Mechanism	Exploitation Status
Low	None (Direct Concatenation)	Fully Vulnerable
Medium	Blacklist (;, &&)	Bypassed via Pipe () or Nested Chaining
High	Expanded Blacklist with Syntax Flaws	Bypassed via Space Omission (cat)
Impossible	Strict Whitelisting & API Isolation	Secure (Unhackable)

To eliminate command injection entirely, developers must avoid system shell invocation functions like shell_exec() or system(). Implementing strict input validation whitelists (e.g., verifying that input matches a strict IPv4 regular expression pattern) ensures that malicious syntax is rejected at the entry point.